

Lido Lend
Deployment Verification

Prepared by Alix40 - Cyfrin

Stage 1 - Core

Ethereum mainnet

Verification block 26385552

November 11, 2026

Contents

1 Deployment Bytecode Verification - Stage 1 (Core) 2

2 Audited source alignment 3

3 Reproducible verification process 3

4 Verification results 5

5 Supplementary core proxy checks 5

6 Supplementary dependency deployment verification 6

7 Deployment Bytecode Verification - Stage 2 (Pending) 7

1 Deployment Bytecode Verification - Stage 1 (Core)

On November 10, 2026, following the deployment of Lido Lend's core contracts to Ethereum mainnet, Cyfrin verified that the runtime bytecode of the 14 Lido core contracts listed below in the client's deployment list matches code compiled locally from the stated deployment commit, 9c0ba345. The comparison includes immutable values and the trailing compiler metadata; no runtime bytes were excluded from the final comparison.

The verification was performed against finalized Ethereum mainnet block **26385552**, timestamp **2026-11-10 15:57:11 UTC**, block hash `0x153f5ca2c0f185ace443c015743d1a9cd7b5001fb14ce211415baeda15261c5a` (chain ID `1`). The block hash and the 14 listed contracts' runtime bytecodes were cross-checked using two RPC endpoints.

Contract	Deployed address
`LidoLend` (implementation)	`0xa40Bd2281F54A2D3845F9535b74D2abD946E1911`
`LidoLendOwner`	`0x85378a1073E8f338e11A97E090E47e2F0FdD3D5a`
`IrmRouter` (implementation)	`0x683eA34c29658fb6cDeCc92Eb4B4e7f766979868`
`AdaptiveCurveIrm`	`0xf6f1565F4fD3d29E6bDAb86764A50BCde381422f`
`IrmConfigFactory`	`0xE7ce129Eb67970C4d71e1456ad50f84B43C35496`
`RiskStewardDashboard`	`0xC3C241bC213EF3ba16234A7c689999209cE515D7`
`LidoLendMarketFactory`	`0xBe1C1eaBda3eD8a93D3CE3246cf4EeF8111c76C7`
`LidoLendMarketController` (template implementation)	`0x69136Ab2be05194C0b2773f6E2Cb944bCeb1D6b8`
`LidoLendERC4626` (template implementation)	`0x8549736900BF3e643D7660E6DfDd09B0FBa2E286`
`GuardianSignals` (template implementation)	`0x66aFC8Bb3aB2f01127fDdEb05425bAD77923091f`
`EthereumGeneralAdapter1`	`0x0C6151F0205A6Edf143b42C4f7B4E97a8f099260`
`LidoLendVaultAdapter`	`0xb1A64663B5390Ecf1a582df7e19d0a6753d1B84A`
`MorphoExitBook`	`0xAd28EFa8f2ef56a68b0Eb6836f4aE1d2D6459Ecd`
`Erc4626ExitBook`	`0x0d2580e4c9b20507eDFAeFC7777d760F12Ad55D4`

The listed LidoLend and IrmRouter addresses are implementation contracts. The controller, ERC-4626 vault and GuardianSignals entries are template implementations. Verification of those templates does not establish that a market using them has been deployed or activated.

2 Audited source alignment

The client identified the following final audited public commits:

Repository	Final audited public commit	Dependency revision used by the deployment checkout
lend	`4d55f1dd98a0f7712543965c66e03c963a004263`	`422d589f9d098124249b54b55306884165833207`
lend-irm	`97af4c39c51f1e023d8025319f34a1658036066c`	`566d31f7dfdf83d07a6593f641f7474c263a7f89`
lend-markets	`23416c43c4038ea3c00a13d6203949c8f88905cc`	deployment commit `9c0ba34580ab5cff4354be8cb701419fb02b65d5`

Cyfrin recomputed the keccak256 digests of all **53 in-scope files** at the deployment checkout: 25 markets files, 12 core files and 16 interest-rate-model files. Every file matches the digest recorded in §6 of the audit report. The aggregate root also matches:

```
0x84be32b38acdf912b6ae7fbaabe235398f03d682394d121dcdc41af042b4ecc4
```

The `lend-markets/src/` tree is unchanged between the final audited public markets commit and the deployment commit. The deployment tree uses the older `lib/lido-lend/` and `lib/lido-lend-irm/` directory names; this naming difference does not change the file contents. The scope of this source-alignment conclusion is the 53 files covered by the report checksums, not a new audit of dependencies or other files.

3 Reproducible verification process

1. From the verification-materials directory, prepare the deployment checkout and its pinned dependencies at the path expected by the helper scripts:

```
mkdir -p repositories
git clone https://github.com/lidofinance/lend-markets.git repositories/lend-markets
git -C repositories/lend-markets fetch origin 9c0ba34580ab5cff4354be8cb701419fb02b65d5
git -C repositories/lend-markets checkout --detach 9c0ba34580ab5cff4354be8cb701419fb02b65d5
git -C repositories/lend-markets -c 'url.https://github.com/.insteadOf=git@github.com:' \
  submodule update --init --recursive
```

The HTTPS rewrite applies to that command only and permits fetching nested dependencies whose recorded URLs use SSH. The accompanying `README.md` lists the required tools, compiler executable names and Python dependencies. Run the helper commands from the verification-materials directory.

2. Compile the contracts locally using their exact compiler versions and deployment settings:

Contracts	Solidity compiler	EVM target	Optimizer	viaIR	Metadata
`EthereumGeneralAdapter1`	`0.8.28+commit.7893614a`	Cancun	enabled, 200 runs	false	bytecodeHash = none; appendCBOR = true
Other 13 listed contracts	`0.8.36+commit.8a079791`	Cancun	enabled, 200 runs	false	bytecodeHash = none; appendCBOR = true

The deployment checkout leaves the EVM target implicit. Verification therefore used **Cancun explicitly**, as recorded in the verified deployment metadata, rather than relying on the default of the locally installed build tool.

For each contract, the standard-JSON compilation input was assembled from the checked-out Git revisions, including the recorded dependency gitlinks. Every source file in the explorer's compilation bundle was checked byte for byte against its pinned Git blob before the Git-derived content was supplied to the local compiler. Explorer source content was not substituted for the checked-out source.

The standard-JSON inputs and complete compiler outputs are retained in the verification materials. They can be recompiled with the corresponding compiler, for example:

```
solc0.8.36 --standard-json < build/LidoLend-input.json > LidoLend.rebuilt.json
solc0.8.28 --standard-json < build/EthereumGeneralAdapter1-input.json > EthereumGeneralAdapter1.rebuilt.json
```

The accompanying `scripts/build_verified_sources.py` performs this compilation for all 14 listed contracts and records the Git provenance, runtime bytecode, ABI, and compiler-generated `immutableReferences`. No unresolved library link references were present.

3. Fetch the on-chain runtime code at the pinned block and compare it with the local output:

```
cast code 0xa40Bd2281F54A2D3845F9535b74D2abD946E1911 \
  --block 26385552 --rpc-url <archive-mainnet-rpc> > LidoLend.onchain.hex
```

The same operation was performed for every address in the table. The three contracts without immutable runtime fields - `LidoLend`, `IrmConfigFactory` and `GuardianSignals` - match the compiler's unpatched runtime output exactly. For the other 11 contracts, the initial runtime differences occur only at the precise positions declared by that local compiler output's `immutableReferences`. Arbitrary zero bytes were not treated as placeholders, and metadata was not stripped.

4. Verify the creation code, constructor arguments and resulting immutable values:

For all 14 listed contracts, Cyfrin obtained the successful contract-creation transaction and receipt. Each is a direct creation transaction whose receipt identifies the listed contract address. The transaction input is exactly the locally compiled creation bytecode followed by the constructor argument encoding; the arguments also match those displayed by the explorer.

The locally compiled creation code and those on-chain arguments were then evaluated with `eth_call` on a local Anvil fork of block `26385552`, using the original transaction sender and value. This executes each constructor, including the external `wstETH` getter used to derive the adapter's `ST_ETH` immutable. No transaction was sent to mainnet. In every case, the constructor's returned runtime code is **byte-identical to the entire on-chain runtime**, including all immutable values and metadata.

The companion commands, run from the verification-materials directory, are:

```
anvil --fork-url <archive-mainnet-rpc> --fork-block-number 26385552 \
  --chain-id 1 --hardfork cancun --host 127.0.0.1 --port 18545

# In a separate terminal, with Python dependencies installed:
python3 scripts/replay_constructors.py
python3 scripts/verify_saved_results.py
```

4 Verification results

The following hashes are keccak256 of the complete on-chain runtime bytecode. Each equals the hash of the runtime returned by local execution of the corresponding compiled constructor.

Contract	keccak256(on-chain runtime bytecode)	Result
`LidoLend`	`0xf882bc080060c8f2a619c2cd6a18798243fdc8cec893f757fdf33be6ee253123`	byte-identical
`LidoLendOwner`	`0xe878a8ccd518374a253b27602c049c1594a0179ac87d6ddb92cb4caa768130b7`	byte-identical
`IrmRouter`	`0x234c06178f1949759a33dd95b351860a2db9ce81509e02ca7444a07ed3e8a97a`	byte-identical
`AdaptiveCurveIrm`	`0x883a7881d3ca49825fdbfd0c85fed8649517b7445705e6417e99f2b2a90ca3d0`	byte-identical
`IrmConfigFactory`	`0x4b9b843759fb083de73c96f187b136d27c3cdb564c18ff27ce965bb7361db38f`	byte-identical
`RiskStewardDashboard`	`0x056f70ff33c41e7c8a86818b5f7b8a06e7c92e6c5ecb4d7e7ecf9fed30921914`	byte-identical
`LidoLendMarketFactory`	`0xf00672169018c0773fa3b278b311442950c6a3ecaab1b652c5b9df45d46d5d29`	byte-identical
`LidoLendMarketController`	`0x3dd65bcface2718985297052fc6c1a8699f65afdff60b90a03045e2d44179a06`	byte-identical
`LidoLendERC4626`	`0xa62d4eb77ce4b31fcd493c129347ceb31adcb8d7cdc14ffe3c0783cd5bc25d17`	byte-identical
`GuardianSignals`	`0x759b65fd142cd8ed2bb819ff9ab76f81a60a30f18bce79e1bb057fe2b68a24b4`	byte-identical
`EthereumGeneralAdapter1`	`0xd55e084bd2b7768c448709be0e6c6291d94f9cc53b189033a9d5a2c6bd049786`	byte-identical
`LidoLendVaultAdapter`	`0xb0306c273145c8fa059e3c7bcdb4afe5a8bd68718fc6aa6d7f39b51e70667701`	byte-identical
`MorphoExitBook`	`0xc06774590c76dc5f94f4d0abf1b6c176d63a5d524dcb6826392cec44dcc66f23`	byte-identical
`Erc4626ExitBook`	`0xcc74151f3af88ef22034b38f9c225cf54c39e29a1069c8a8b09a402374f0aec5`	byte-identical

5 Supplementary core proxy checks

The constructor constants identify two live proxies that are separate from the implementation addresses above. Cyfrin additionally compiled their `OssifiableProxy` code from the deployment checkout and verified that both complete on-chain runtimes match that compiled output exactly. Their EIP-1967 implementation slots at the same block point to the verified implementations:

Proxy	Proxy address	Implementation at block 26385552
LidoLend proxy	`0x026a770badc7126c7252Ef8d407E5b9C56147491`	`0xa40Bd2281F54A2D3845F9535b74D2abD946E1911`
IrmRouter proxy	`0x81b7cc9523b30429110282d90db2fa90ff443d91`	`0x683eA34c29658fb6cDeCc92Eb4B4e7f766979868`

Both proxy runtimes have keccak256 `0x489816a1b58525daf440b9ea0e39235cbee14f6647a47285dad65bda0c0fe056`. These supplementary checks establish proxy-code identity and the implementation pointers at the pinned block; they do not constitute a review of all governance permissions, initialization state or launch configuration.

Unlisted dependency deployments and a complete review of governance permissions and role configuration are outside this bytecode-verification scope.

6 Supplementary dependency deployment verification

On November 11, 2026, at the client's request, Cyfrin additionally verified the following two dependency deployments at the same Ethereum mainnet block, **26385552**. These contracts were outside the Lido Lend audit scope; this supplementary work verifies deployment bytecode identity and does not constitute a new security audit or extend the 53-file audit checksum scope.

Contract	Deployed address
`Bundler3`	`0x0056b888E90a1b783311351B54B782dE5de61e87`
`CircuitBreaker`	`0xd0A81A130a303237d338A0b5835edcB011Ab3afd`

The updated deployment list identifies these source revisions:

Repository	Client-reported audited commit	Deployed commit
Bundler3	`263f75d3662f38133d5fba1b08c7b07847abfb2a`	`9afc2f4ea32c9dbeba2205a485731b8a98f7ae4c`
CircuitBreaker	`b4b2fbc921b3191560a3fc62d502d4bb98ad99e1`	`6829a5a962ece56564bd9d72d01c29cabf157579`

Both deployed revisions match the dependency gitlinks in the Lido deployment checkout. All compilation sources were read from pinned Git blobs and checked against the explorer's source bundle.

Bundler3 was compiled with **Solidity 0.8.28+commit.7893614a** and CircuitBreaker with **Solidity 0.8.34+commit.80d5c536**. Both builds use Cancun, optimizer enabled with 200 runs, `viaIR = false`, `bytecodeHash = none` and `appendCBOR = true`. No unresolved library link references were present.

The two target source files are byte-identical between their client-reported audited and deployed commits. Bundler3's imported `IBundler3.sol` differs only in documentation. Compiling both source bundles with the deployment settings produced identical target creation and runtime bytecodes.

Two RPC endpoints confirmed the block identity, runtimes and successful direct-creation transactions. Each address was derived from its creator and nonce, and each transaction input equals the local creation code plus the recorded constructor arguments.

Bundler3 has no runtime immutables and matches the unpatched compiler output. CircuitBreaker's five immutables are `ADMIN`, `MIN_PAUSE_DURATION`, `MAX_PAUSE_DURATION`, `MIN_HEARTBEAT_INTERVAL` and `MAX_HEARTBEAT_INTERVAL`. Local constructor execution on the verified Anvil fork with the original arguments, sender and value returned each complete on-chain runtime exactly, including all immutables and metadata.

Contract	keccak256(on-chain runtime bytecode)	Result
`Bundler3`	`0x94f57c1dd1fd8a4d8b095693f49bc71c4c926cba9f5ae01d8e6f3d5cdeac7d66`	byte-identical
`CircuitBreaker`	`0x8d4f4df9319d212c0dc3b1a843a5fcab3225e0f75df5e8c656cad49810b0b02c`	byte-identical

The accompanying `scripts/verify_supplemental_dependencies.py` reproduces these checks and validates the retained raw evidence. See `README.md` for compiler setup and commands.

CircuitBreaker's live pause settings, registrations, heartbeat state and governance suitability were not reviewed. Market-specific deployments remain pending Stage 2.

7 Deployment Bytecode Verification - Stage 2 (Pending)

Stage 2 will cover the market-specific deployments, beginning with WETH/wstETH, once their deployed addresses and deployment commits are supplied. This document will be updated with the Stage 2 verification results after those checks are completed.